Proxy

Definition
Use cases of the Proxy Pattern
Class Diagram
Structure of Proxy Pattern
Implementation

Resources

- Video → 32. All Structural Design Patterns | Decorator, Proxy, Composite, Adapter, Bridge, Facade, FlyWeight
- Video → 13. Proxy Design Pattern Explanation (Hindi) | LLD | System Design Interview Question | Java

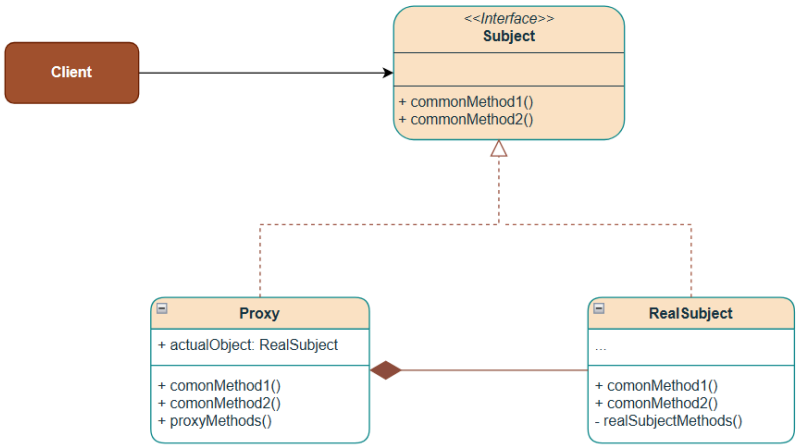
Definition

> ***The Proxy Design Pattern provides a representative or placeholder for another object(real complex object) to control access to it.***

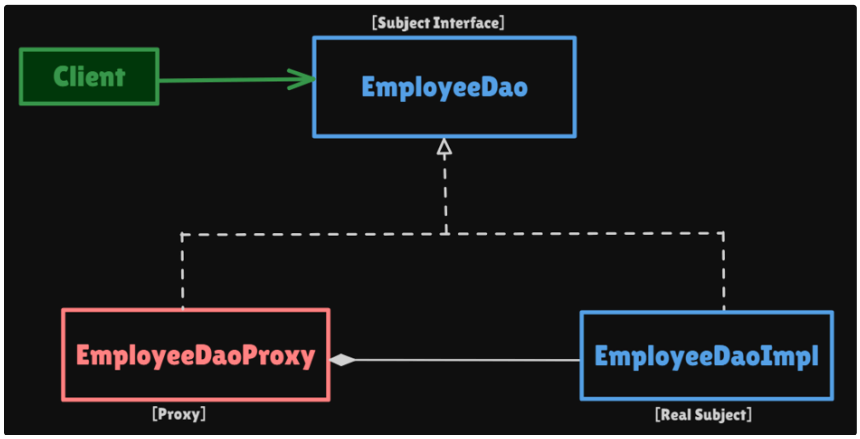
Use cases of the Proxy Pattern

- **Access Control**: Restricts access to sensitive operations based on user permissions.
- **Performance Optimization**: Delays resource-intensive operations like expensive object creation until necessary (lazy loading).
- **Pre/post processing:** Performs additional operations before and after forwarding the request to the actual object, like adding cross-cutting concerns like audit traces (logging & monitoring), without modifying core business logic.
- **Caching**: Can cache results to improve performance.

Class Diagram



Structure of Proxy Pattern



1. **Subject Interface (** `EmployeeDao` **)**
   - Defines the common interface for both `EmployeeDaoImpl` and `EmployeeDaoProxy`.
   - Ensures the proxy can be used anywhere the real object is expected
2. **Real Subject (** `EmployeeDaoImpl` **)**
   - Contains the actual business logic and data. Handles core employee operations.
   - Performs expensive operations like database loading.
3. **Proxy Object (** `EmployeeDaoProxy` **)**
   - Maintains a reference to the Real Subject(`EmployeeDaoImpl`).
   - Controls access through permission checks.
   - Can be modified further to implement lazy loading, logging and etc.
4. **Client (** `EmployeeManagement` **)**
   - Client is performing operations on the subject without being aware of whether it's a Real/Proxy object

Implementation

```java
public interface EmployeeDao {
    void getEmployeeInfo(int empID);
    void createEmployee(EmployeeDo obj);
  }
```

```java
public class EmployeeDaoImpl implements EmployeeDao {
    public void getEmployeeInfo(int empID) {
        System.out.println("Fetching employee info for ID: " + empID);
    }

    public void createEmployee(EmployeeDo obj) {
        System.out.println("Creating employee: " + obj);
    }
}
```

```java
 public class EmployeeDaoProxy implements EmployeeDao {

     private EmployeeDao empDaoObj;
     private String clientRole;

     public EmployeeDaoProxy(String clientRole) {
         empDaoObj = new EmployeeDaoImpl();
         this.clientRole = clientRole;
     }

```

```java
    @Override
    public void getEmployeeInfo(int empID) {
        if (clientRole.equals("ADMIN") || clientRole.equals("USER")) {
            empDaoObj.getEmployeeInfo(empID);
        } else {
            throw new RuntimeException("Access Denied");
        }
    }

    @Override
    public void createEmployee(EmployeeDo obj) {
        if (clientRole.equals("ADMIN")) {
            empDaoObj.createEmployee(obj);
        } else {
            throw new RuntimeException("Access Denied");
        }
    }
}
```

```java
// Client
public class EmployeeManagement {
    public static void main(String[] args) {
        System.out.println("===== Proxy Design Pattern =====");


        EmployeeDao userProxyObj = new EmployeeDaoProxy("USER");
        userProxyObj.getEmployeeInfo(); //access granted
        userProxyObj.createEmployee(new EmployeeDo()); //access denied
    }
}
```

The Proxy pattern is particularly useful in enterprise applications where security, performance, and monitoring are critical concerns.